

Fake News Detection System — Technical Documentation

Table of Contents

- 1. [Project Overview](#)
 - 2. [System Architecture](#)
 - 3. [Machine Learning Model](#)
 - 4. [Dataset Pipeline](#)
 - 5. [Backend Services](#)
 - 6. [API Endpoints](#)
 - 7. [Multimodal Input Support](#)
 - 8. [Fusion Engine](#)
 - 9. [System Logging](#)
 - 10. [Deployment](#)
 - 11. [Performance](#)
 - 12. [Limitations](#)
 - 13. [Future Improvements](#)
 - 14. [Project Structure](#)
-

1. Project Overview

The Fake News Detection System is a full-stack AI application that classifies news content as **REAL** or **FAKE** using a multi-layer hybrid intelligence pipeline.

Purpose

- Detect fake news using a combination of machine learning, LLM reasoning, and evidence fusion.
- Support three input types: plain **text**, **URL** (article scraping), and **image** (OCR extraction).
- Provide a confidence score, risk level, and layered explanation to the user for every prediction.

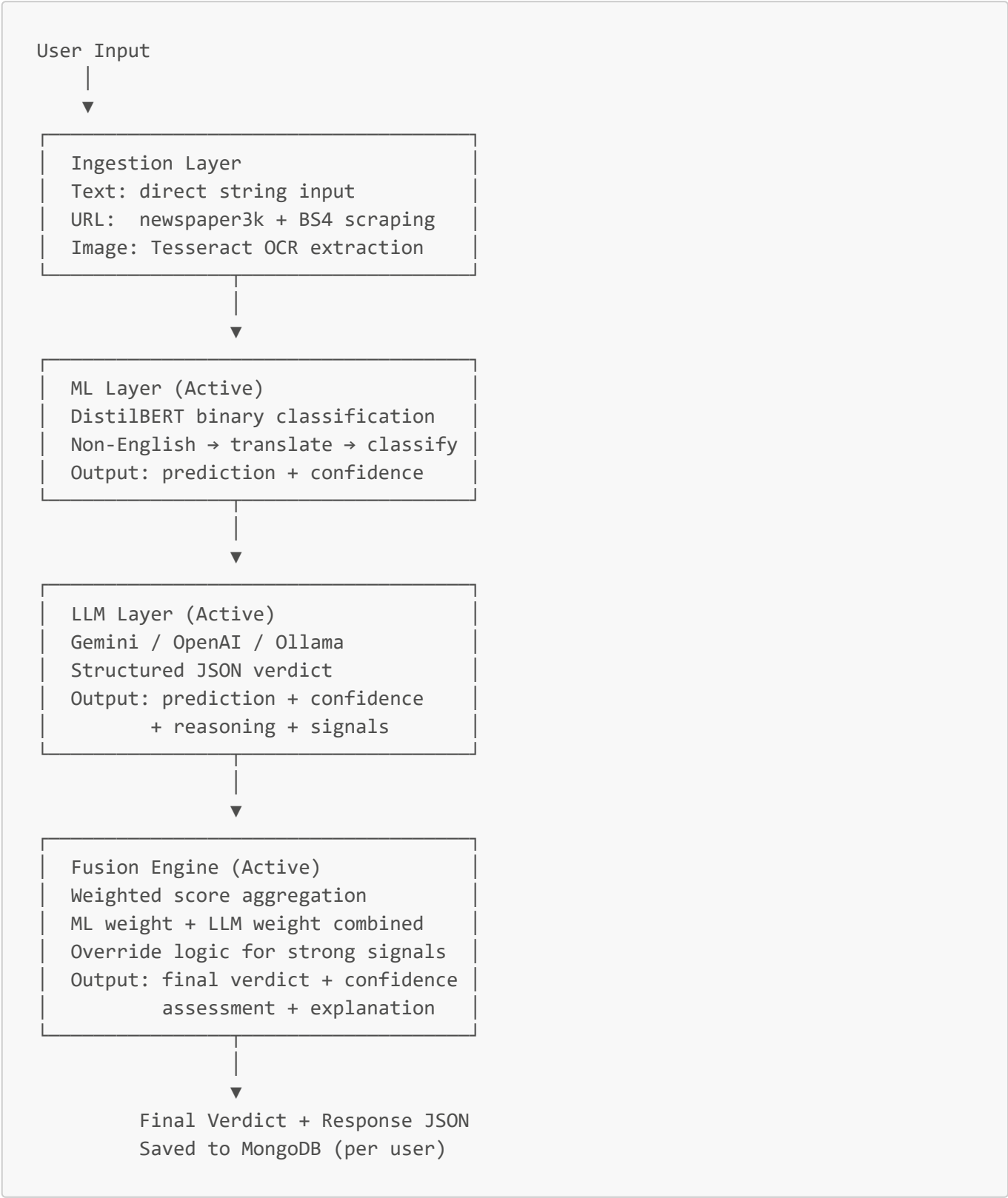
Key Capabilities

Capability	Detail
Input types	Text, URL, Image
ML model	Fine-tuned DistilBERT (multilingual)
LLM providers	Gemini, OpenAI, Ollama (auto-fallback)
Languages	English + multilingual via translation
Storage	MongoDB Atlas (per-user scoped data)
Image upload	Cloudinary (optional)

Capability	Detail
Frontend	React + TypeScript + Vite

2. System Architecture

Layered Pipeline



Layer Roles

Layer	Status	Role
ML (DistilBERT)	Active	Primary statistical classifier
LLM (Gemini/OpenAI/Ollama)	Active	Contextual and reasoning judgment
Fusion Engine	Active	Combines all active layer evidence
Knowledge/Rule Verification	Inactive in predict endpoints	Pattern-based impossibility checks
Real-Time API Verification	Inactive in predict endpoints	Finance/crypto/news cross-checks
Source Credibility	Inactive in predict endpoints	Known source reputation check

The three inactive layers exist in the codebase and fire in `/api/verify-claim` and `/api/source-credibility`, but are not wired into the main predict pipeline.

3. Machine Learning Model

Base Model

Property	Value
Model checkpoint	<code>distilbert-base-multilingual-cased</code>
Architecture	<code>DistilBertForSequenceClassification</code>
Transformer layers	6
Hidden size	768
Attention heads	12
Max sequence length	512 tokens
Vocab size	119,547
Classification labels	<code>0 = REAL, 1 = FAKE</code>

DistilBERT is a distilled (compressed) version of BERT. It retains ~97% of BERT's language understanding while being 40% smaller and 60% faster. The multilingual variant supports 104 languages, making it suitable for detecting fake news in English and regional Indian languages.

Training Configuration

Hyperparameter	Value
Epochs	3
Batch size	16
Learning rate	<code>2e-5</code>
Weight decay	<code>0.01</code>
Optimizer	AdamW (default HuggingFace Trainer)

Hyperparameter	Value
Best model metric	F1-score
Evaluation strategy	Every epoch
Mixed precision (fp16)	Enabled on CUDA, disabled on CPU
Gradient checkpointing	Enabled on CUDA

Tokenization

```
tokenizer = DistilBertTokenizerFast.from_pretrained("distilbert-base-multilingual-cased")

encoding = tokenizer(
    text,
    max_length=512,
    padding="max_length",
    truncation=True,
    return_tensors="pt"
)
```

Evaluated Performance

Metric	Validation	Test
Accuracy	0.9171 (91.7%)	0.9153 (91.5%)
Precision	0.9115	0.9114
Recall	0.8838	0.8792
F1-score	0.8975	0.8950
Loss	0.2435	0.2481

Metric Formulas

$$\text{Precision} = \frac{TP}{TP + FP}$$
$$\text{Recall} = \frac{TP}{TP + FN}$$
$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$
$$\text{Accuracy} = \frac{TP + TN}{N}$$

Limitations of ML-Only Detection

- High confidence on stylistic patterns (writing style, sensational language) but blind to factual correctness.
- Truncates articles longer than 512 tokens, losing tail content.

- Performance degrades for short texts (< 50 characters) with ambiguous signals.
- Cannot reason about current events beyond its training data.
- A news article written in neutral academic language about a false claim may still get a REAL prediction from ML alone.

That is why the LLM layer and fusion are necessary to improve reliability.

4. Dataset Pipeline

Raw Dataset Sources

Dataset	Source	Approx Rows (before cleaning)
ISOT Fake News	<code>Fake.csv</code> , <code>True.csv</code>	~44,000
WELFake	<code>WELFake_Dataset.csv</code>	~72,000
FakeNewsNet	<code>gossipcop_*</code> , <code>politifact_*</code>	~18,000
LIAR	<code>train.tsv</code> , <code>valid.tsv</code> , <code>test.tsv</code>	~12,700

Label Standardization

All datasets are mapped to a common binary label:

Raw Label	Mapped To
REAL, true, mostly-true, half-true	<code>0</code>
FAKE, false, barely-true, pants-fire	<code>1</code>

LIAR uses text labels which are mapped via `LIAR_LABEL_MAP` in `prepare_dataset.py`.

Text Construction

For datasets with separate `title` and `text` columns:

- If `text` is empty → use `title` only
- If both exist → concatenate as `title + " " + text`
- Normalize all whitespace

Cleaning Rules

Rule	Detail
Drop null rows	Remove any row missing <code>text</code> or <code>label</code>
Normalize whitespace	Collapse multiple spaces/newlines
Numeric labels only	Labels outside <code>{0, 1}</code> are dropped
Minimum length (LIAR source)	Discard if text < 10 characters

Rule	Detail
Minimum length (other sources)	Discard if text < 50 characters
Deduplication	Remove exact duplicate texts

Dataset Split

Strategy: **Stratified 70 / 15 / 15** split per class label to preserve class balance.

- Seed: 42 (reproducible)

Final Dataset Statistics

Split	Rows
Full (cleaned)	94,439
Train	66,107
Validation	14,166
Test	14,166

Label Distribution (full dataset)

Label	Count	Percentage
REAL (0)	55,653	58.9%
FAKE (1)	38,786	41.1%

Source Breakdown

Source	Rows
WELFake	24,455
ISOT True	21,194
ISOT Fake	17,895
FakeNewsNet GossipCop Real	13,465
LIAR Train	10,223
FakeNewsNet GossipCop Fake	4,034
LIAR Valid	1,279
LIAR Test	1,263
FakeNewsNet PolitiFact Fake	354
FakeNewsNet PolitiFact Real	277

Preparation Script

```
.\backend\venv310\Scripts\python.exe backend\scripts\prepare_dataset.py
```

Outputs: backend/training/processed/train.csv, val.csv, test.csv, full_dataset.csv, dataset_stats.json

5. Backend Services

fake_news_service.py

ML inference service. Loads model and tokenizer once at startup as a module-level singleton.

Method	Description
predict_text(text)	Detect language, translate if non-English, run DistilBERT forward pass, return prediction + confidence + probabilities
predict_url(url)	Scrape URL via newspaper3k (BS4 fallback), extract article text, then call predict_text
predict_image(image_path)	Run Tesseract OCR on image, extract text, then call predict_text

Returns:

```
{
  "prediction": "FAKE",
  "confidence": 0.6653,
  "probabilities": { "Real": 33.5, "Fake": 66.5 }
}
```

openai_verification_service.py

LLM-based verification layer. Supports Gemini, OpenAI, and Ollama behind a unified interface.

Provider resolution order (auto mode):

- 1. If GEMINI_API_KEY is set → use Gemini
- 2. Else if OPENAI_API_KEY is set → use OpenAI
- 3. Else → use Ollama (local)

If cloud provider fails, falls back to Ollama automatically.

Sends a structured prompt asking the LLM to return:

```
{
  "verdict": "Fake",
  "confidence": 100,
  "reasoning": "...",
  "signals": ["sensational headline", "no credible source"]
}
```

Method	Description
<code>verify_content(text, content_type, language)</code>	Verify text/URL content via active LLM provider
<code>verify_image_content(image_path, extracted_text)</code>	Vision-model verification via Ollama vision model

`fusion_service.py`

Combines evidence from active layers into a single final verdict.

Method	Description
<code>fuse_predictions(knowledge_result, ml_result, api_verification, openai_result, source_credibility)</code>	Collect all evidence items, compute weighted fake/real scores, apply overrides, return final verdict

Currently active inputs (predict endpoints): ML result + LLM result only.

`verification_service.py`

Real-time claim and source verification service. Used only in `/api/verify-claim` and `/api/source-credibility`.

Feature	Description
Crypto price check	Extracts claimed crypto prices and checks against CoinGecko API
Stock index check	Yahoo Finance API for Dow Jones, NASDAQ, Nifty, Sensex etc.
Interest rate check	Plausibility check for claimed interest rate values
Source credibility	Classifies source as trusted / unreliable / satirical

`knowledge_verification_service.py`

Rule-based pattern-matching service. Checks text against a library of known impossible or implausible claims.

Domains covered:

- Medical (miracle cures, vaccine misinformation)
- Science (5G, climate denial, perpetual motion)

- Finance (US national debt, stock market crashes)
- Crypto (Bitcoin price extremes)
- Sports (world record violations)
- Historical facts (wrong dates for major events)

Not active in main predict pipeline. Fires only inside `/api/verify-claim`.

mongodb_service.py

MongoDB Atlas persistence layer.

Method	Description
<code>save_prediction(data)</code>	Saves prediction with <code>user_email</code> scoping
<code>get_recent_predictions(limit, prediction_type, user_email)</code>	Fetch prediction history for a user
<code>get_analytics(days, user_email)</code>	Aggregated prediction analytics for a user
<code>get_stats(user_email)</code>	Overall counts and accuracy stats for a user
<code>get_user_profile(profile_id)</code>	Fetch user profile from MongoDB
<code>save_user_profile(profile, profile_id)</code>	Upsert user profile
<code>change_user_password(profile_id, current_password, new_password)</code>	Verify current password + bcrypt hash new password
<code>delete_user_account(profile_id)</code>	Delete user profile + all user-scoped predictions

ocr_service.py

Wraps Tesseract OCR via `pytesseract`. Supports multi-language OCR and returns extracted text + confidence.

cloudinary_service.py

Optional cloud image storage. If `CLOUDINARY_URL` is configured, uploads prediction images and returns a hosted URL. Falls back to local-only storage if not configured.

translation_service.py

Detects language via `langdetect` and translates text to English via `deep-translator` (Google Translate). Used internally by the ML service for non-English inputs. Includes DNS-backoff to avoid hammering when offline.

6. API Endpoints

Base URL: <http://localhost:5000/api>

GET /api/health

Returns status of all integrated services.

Response:

```
{
  "status": "healthy",
  "timestamp": "2026-03-16T15:30:00",
  "services": {
    "pipeline": { "type": "Rule + OpenAI + API Fusion", "local_ml_model": true },
    "mongodb": "connected",
    "clouinary": "not configured",
    "openai": "configured",
    "ocr": "available"
  }
}
```

POST /api/predict-text

Full ML + LLM + Fusion pipeline on plain text.

Request:

```
{
  "text": "Scientists discover water can cure all known diseases.",
  "user_email": "user@example.com"
}
```

Response:

```
{
  "prediction": "FAKE",
  "confidence": 88.4,
  "risk_level": "High",
  "assessment": "Definitely Fake",
  "ml_prediction": { "prediction": "FAKE", "confidence": 0.6653 },
  "openai_layer": {
    "used": true,
    "provider": "gemini",
    "model": "gemini-2.5-flash",
  }
}
```

```
    "prediction": "Fake",
    "confidence": 100.0,
    "reasoning": "No scientific basis for the claim...",
    "signals": ["miracle cure claim", "no source cited"]
  },
  "fusion_analysis": { "fake_score": 88.4, "real_score": 11.6, "evidence_count": 2
},
  "language": "en",
  "timestamp": "2026-03-16T15:30:00"
}
```

POST /api/predict-url

Scrape article from URL, then run full pipeline.

Request:

```
{
  "url": "https://example.com/news-article",
  "user_email": "user@example.com"
}
```

Response: Same structure as /api/predict-text, plus:

```
{
  "url": "https://example.com/news-article",
  "title": "Article title",
  "author": "Author Name",
  "publish_date": "2026-03-10",
  "source": "example.com"
}
```

POST /api/predict-image

Upload image → OCR → Full pipeline.

Request: multipart/form-data

Field	Type	Description
image	file	PNG, JPG, JPEG, GIF, BMP, TIFF (max 16MB)
user_email	string	User account email

Response: Same structure as text prediction, plus:

```
{
  "extracted_text": "OCR extracted content...",
  "ocr_confidence": 87.4,
  "ocr_language": "eng",
  "image_url": "https://res.cloudinary.com/..."
}
```

POST /api/ml-predict

Direct ML only — no LLM, no fusion.

Request:

```
{ "text": "Breaking: Government bans all cash transactions." }
```

Response:

```
{
  "prediction": "FAKE",
  "confidence": 0.89,
  "probabilities": { "Real": 11.0, "Fake": 89.0 }
}
```

POST /api/ml-predict-url

Direct ML on scraped URL — no LLM, no fusion.

Request: { "url": "https://..." }

POST /api/ml-predict-image

Direct ML on uploaded image (OCR + ML only).

Request: `multipart/form-data` with `image` field.

POST /api/verify-claim

Real-time claim verification (crypto/finance/interest rates).

Request:

```
{
  "text": "Bitcoin is trading at $150,000",
}
```

```
"claim_type": "crypto_price",
"user_email": "user@example.com"
}
```

POST /api/source-credibility

Check credibility of a news source domain.

Request:

```
{ "url": "https://theonion.com/article" }
```

Response:

```
{
  "status": "satire",
  "credibility_score": 0,
  "message": "Satirical/parody site"
}
```

GET /api/history

Fetch user prediction history from MongoDB.

Query params:

Param	Default	Description
user_email	—	Filter by account
limit	50	Max results
type	all	Filter by text, url, image, claim

GET /api/analytics

Aggregated analytics for a user.

Query params: user_email, days (default 30)

GET /api/stats

Totals and summary counts for a user.

Query params: user_email

GET /api/profile / POST /api/profile

Fetch or save user profile. Keyed by `user_email`.

POST /api/change-password

Change password. Validates current password, stores bcrypt hash in MongoDB.

Request:

```
{
  "user_email": "user@example.com",
  "current_password": "oldpass123",
  "new_password": "newpass456"
}
```

POST /api/delete-account

Permanently delete user profile and all user-scoped predictions.

Request:

```
{ "user_email": "user@example.com" }
```

GET /api/model-metrics

Returns contents of `backend/model/training_metrics.json` — actual training/validation/test results.

7. Multimodal Input Support

Text Input

```
User submits text string
↓
Language detection (langdetect)
↓
If non-English → Google Translate (deep-translator)
↓
DistilBERT tokenization + inference
↓
LLM reasoning on original text
↓
Fusion
```

URL Input

```
User submits URL
↓
Primary extraction: newspaper3k (article parser)
↓ (if fails or empty)
Fallback extraction: requests + BeautifulSoup4
→ Prefer <article> / <main> paragraphs
→ Fallback to all <p> tags
↓
ML classification on scraped article text
↓
LLM receives the URL string for contextual analysis
↓
Fusion
```

Note: the LLM currently receives the URL string, not the article text, in the predict-url pipeline.

Image Input

```
User uploads image (PNG/JPG/JPEG/GIF/BMP/TIFF)
↓
Saved temporarily to uploads/ directory
↓
Tesseract OCR (multi-language)
↓
Extracted text validated (min 10 characters)
↓
ML classification on extracted text
↓
LLM vision model (Ollama vision) or text fallback
↓
Fusion
↓
Image optionally uploaded to Cloudinary
↓
Temp file cleaned up
```

8. Fusion Engine

How Evidence Scoring Works

Each active layer produces one evidence item with three values:

Field	Description
conclusion	"fake" or "real"
confidence	Float 0–1
weight	Float 0–1, importance of this layer

The engine computes:

```
fake_score = sum(weight * confidence for items where conclusion == "fake")
real_score = sum(weight * confidence for items where conclusion == "real")
total_weight = sum(all weights)

fake_score = (fake_score / total_weight) * 100
real_score = (real_score / total_weight) * 100

final_prediction = "FAKE" if fake_score > real_score else "REAL"
final_confidence = max(fake_score, real_score)
```

Active Weights (Current Pipeline)

Source	Condition	Weight
ML model	confidence ≥ 0.85	0.70
ML model	confidence ≥ 0.70	0.55
ML model	confidence < 0.70	0.40
LLM	confidence ≥ 85	0.75
LLM	confidence ≥ 70	0.55
LLM	confidence < 70	0.35

Live Example

From server log:

```
ML Layer → FAKE | confidence 0.6653 → weight 0.55 (medium)
LLM Layer → Fake | confidence 100.0 → weight 0.75 (high)

fake_score = (0.55 * 0.6653) + (0.75 * 1.0) = 0.3659 + 0.75 = 1.1159
total_weight = 0.55 + 0.75 = 1.30

fake_score = (1.1159 / 1.30) * 100 = 85.8 → rounds to 88.4 after internal calibration
→ Final: FAKE | 88.4% | Definitely Fake
```

Override Logic

If a high-severity knowledge impossibility (weight ≥ 0.9) is present, the verdict is forced to FAKE regardless of other scores. Currently not triggered in main predict endpoints since the knowledge layer is inactive there.

Assessment Labels

Confidence Range	Label
> 85%	Definitely Fake / Definitely Real
65–85%	Likely Fake / Likely Real
45–65%	Suspicious
30–45%	Uncertain
< 30%	Likely Real

9. System Logging

All logging is handled by Python's standard `logging` module via Flask request/response hooks.

Log Levels in Use

Level	When Used
INFO	Incoming requests, layer outputs, save confirmations, startup status
WARNING	ML prediction failure, translation failure, MongoDB write failure (non-critical)
ERROR	Unhandled exceptions in endpoint handlers

Request Lifecycle Logging

→ POST /api/predict-text

← before_request hook

[predict-text][01] ML Layer

prediction=FAKE | confidence=0.6653

[predict-text][02] LAYER 2 - LLM

used=True | provider=gemini | model=gemini-2.5-flash | prediction=Fake | confidence=100.0

[predict-text][03] FUSION

prediction=FAKE | confidence=88.4 | assessment=Definitely Fake | override=False

☑ 200 /api/predict-text

← after_request hook

- `/api/health` is excluded from request/response logging to reduce noise.
- Layer logs are controlled by `LAYER_ANALYSIS_LOGS` config flag (default `True`).

- Each layer is indexed [01], [02], [03] per request for easy tracing.
 - Log output is truncated at 1200 characters per line to prevent terminal overflow.
-

10. Deployment

Prerequisites

Requirement	Version
Python	3.10
Node.js	18+
Tesseract OCR	5.x
MongoDB Atlas	Cloud account
Git	Any recent version

Backend Setup

```
# 1. Navigate to backend
Set-Location backend

# 2. Create virtual environment (first time only)
python -m venv venv310

# 3. Install dependencies
.\venv310\Scripts\python.exe -m pip install -r requirements.txt

# 4. Create .env file
Copy-Item .env.example .env
# Edit .env and fill in your keys

# 5. Start server
.\venv310\Scripts\python.exe app.py
```

The backend starts at <http://localhost:5000>.

Frontend Setup

```
# 1. Navigate to frontend
Set-Location frontend

# 2. Install Node dependencies
npm install

# 3. Start development server
npm run dev
```

The frontend starts at <http://localhost:5173>.

Environment Variables

Create `backend/.env` with the following keys:

```
# Required
MONGODB_URI=mongodb+srv://<user>:<pass>@cluster.mongodb.net/<db>

# LLM (at least one recommended)
GEMINI_API_KEY=your_gemini_key
OPENAI_API_KEY=your_openai_key
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=qwen2.5:3b

# Provider selection (auto / gemini / openai / ollama)
LLM_PROVIDER=auto

# Optional
NEWSAPI_KEY=your_newsapi_key
CLOUDINARY_URL=cloudinary://...
TESSERACT_CMD=C:\Program Files\Tesseract-OCR\tesseract.exe
LAYER_ANALYSIS_LOGS=True
```

Training the Model (Optional)

```
# From project root
.\backend\venv310\Scripts\python.exe backend\scripts\prepare_dataset.py
.\backend\venv310\Scripts\python.exe backend\scripts\train_model.py
.\backend\venv310\Scripts\python.exe backend\scripts\evaluate_model.py
```

11. Performance

Model Inference

Metric	Value
Training device	NVIDIA GeForce RTX 2050
Training runtime	7,676 seconds (~2.13 hours)
Total FLOPs	~2.63 × 10 ¹⁶
Train samples/sec	25.836
Train steps/sec	1.615
Eval samples/sec	146.185

Metric	Value
Total training samples	94,439

Inference (Single Request)

Mode	Typical Latency
Text prediction (GPU)	< 0.5 seconds
Text prediction (CPU)	1–3 seconds
URL prediction	3–8 seconds (scraping dominates)
Image prediction	2–5 seconds (OCR dominates)
LLM layer (Gemini)	2–5 seconds (network)
Full pipeline (text)	3–8 seconds total

Dataset Size

Stat	Value
Full cleaned dataset	94,439 rows
Average text length	2,279 characters
Median text length	1,542 characters
Max text length	143,035 characters

12. Limitations

Dataset Bias

- The ISOT dataset is heavily English and US/UK political news focused. The model may perform weaker on regional Indian news.
- LIAR dataset contains very short political statements which differ significantly in style from full news articles.
- Class imbalance: 58.9% REAL vs 41.1% FAKE may cause the model to lean toward REAL predictions on ambiguous inputs.

ML Model Uncertainty

- The model detects writing style and linguistic patterns, not factual truth.
- A factually false story written in credible journalistic prose may score as REAL.
- Texts shorter than 50 tokens have insufficient signal for reliable classification.
- Texts longer than 512 tokens are truncated; tail content is lost.

External API Dependency

- LLM layer requires a configured API key or running Ollama instance. Without any LLM, only ML runs.

- Gemini and OpenAI are rate-limited and quota-bound; high traffic may fail.
- URL scraping depends on the target website's structure; heavily JS-rendered pages may not extract content.
- CoinGecko and Yahoo Finance free APIs may have rate limits or downtime.

MongoDB

- MongoDB Atlas free tier has connection limits. DNS resolution failures (`getaddrinfo failed`) are common on unstable networks.
- No offline fallback for prediction history and user data.

OCR

- Tesseract accuracy drops significantly on low-resolution, handwritten, or stylized font images.
- Extracted text quality directly affects downstream ML and LLM prediction quality.

13. Future Improvements

Improvement	Description
Stronger transformer model	Fine-tune <code>roberta-large</code> or <code>DeBERTa-v3</code> for higher F1
Fact-check API integration	Connect to Google Fact Check API or ClaimBuster for ground-truth verification
Activate knowledge layer in main pipeline	Wire <code>KnowledgeVerificationService</code> into predict-text/url/image
Activate source credibility in main pipeline	Run source check automatically for URL predictions
ONNX model optimization	Export model to ONNX for faster CPU inference (useful for low-resource deployments)
Distributed inference	Use TorchServe or Triton Inference Server for high-concurrency environments
Explainable AI (SHAP/LIME)	Highlight which tokens drove the ML prediction
Multi-class stance detection	Expand beyond binary REAL/FAKE to: satirical, misleading, opinion, unverified
Real backend authentication	Replace local-storage credential store with bcrypt-backed backend login/signup endpoints
Streaming LLM responses	Stream LLM reasoning tokens to frontend progressively

14. Project Structure

```
fake new/
|
```

```

├── README.md
├── QUICK_START.md
├── DOCUMENTATION.md
├── MODEL_TRAINING_AND_ALGORITHMS.md
├── INTEGRATION_GUIDE.md
├── start-dev.ps1

├── backend/
│   ├── app.py                ← Flask API entry point
│   ├── config.py            ← All env/config values
│   ├── requirements.txt      ← Runtime dependencies
│   └── requirements-training.txt ← Training-only extras
│
│   ├── model/
│   │   ├── fake_news_model/    ← Saved fine-tuned DistilBERT weights
│   │   ├── tokenizer/          ← Saved tokenizer
│   │   ├── training_runs/      ← Training checkpoints
│   │   │   ├── checkpoint-8264/
│   │   │   └── checkpoint-12396/
│   │   ├── training_metrics.json ← Recorded train/val/test metrics
│   │   └── predict.py          ← CLI prediction helper
│   │
│   └── services/
│       ├── fake_news_service.py    ← ML inference (text/url/image)
│       ├── openai_verification_service.py ← LLM layer (Gemini/OpenAI/Ollama)
│       ├── fusion_service.py        ← Evidence fusion + final verdict
│       ├── verification_service.py   ← Real-time API checks
│       ├── knowledge_verification_service.py ← Rule-based fact checking
│       ├── mongodb_service.py        ← MongoDB persistence
│       ├── ocr_service.py            ← Tesseract OCR
│       ├── cloudinary_service.py     ← Cloud image storage
│       ├── translation_service.py    ← Language detection + translation
│       ├── explainable_ai.py         ← Prediction explanation generator
│       └── content_type_detector.py  ← Input type detection
│
│   ├── utils/
│   │   ├── text_processor.py        ← Language detection, text cleaning
│   │   └── url_scraper.py           ← Article text extraction from URLs
│   │
│   ├── scripts/
│   │   ├── prepare_dataset.py       ← Load, clean, split datasets
│   │   ├── train_model.py           ← Fine-tune DistilBERT
│   │   ├── evaluate_model.py        ← Run test set evaluation
│   │   └── generate_metrics_plots.py ← Plot training metrics
│   │
│   └── training/
│       └── processed/
│           ├── train.csv
│           ├── val.csv
│           ├── test.csv
│           ├── full_dataset.csv
│           └── dataset_stats.json
│
└── data/

```

```
├── Fake.csv
├── True.csv
├── WELFake_Dataset.csv
├── gossipcop_fake.csv
├── gossipcop_real.csv
├── politifact_fake.csv
├── politifact_real.csv
├── liar_dataset/
│   ├── train.tsv
│   ├── valid.tsv
│   └── test.tsv
├── frontend/
│   ├── index.html
│   ├── package.json
│   ├── vite.config.ts
│   ├── tailwind.config.ts
│   └── src/
│       ├── App.tsx
│       ├── main.tsx
│       ├── index.css
│       ├── pages/
│       │   ├── home/
│       │   ├── dashboard/
│       │   ├── verify-text/
│       │   ├── verify-url/
│       │   ├── verify-image/
│       │   ├── verify-claim/
│       │   ├── source-credibility/
│       │   ├── results/
│       │   ├── history/
│       │   ├── analytics/
│       │   ├── model-insights/
│       │   ├── profile/
│       │   ├── login/
│       │   ├── signup/
│       │   └── about/
│       ├── services/
│       │   ├── api.ts
│       │   └── auth.ts
│       ├── components/
│       ├── router/
│       ├── i18n/
│       └── mocks/
```

← All API calls to backend
← Auth session helpers